Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3639477.3639744

RESEARCH-ARTICLE

Automated Security Findings Management: A Case Study in Industrial DevOps

MARKUS VOGGENREITER, Siemens AG, Munich, Bayern, Germany

FLORIAN ANGERMEIR, Fortiss GmbH, Munich, Bayern, Germany

FABIOLA MOYÓN, Siemens AG, Munich, Bayern, Germany

ULRICH SCHÖPP, Fortiss GmbH, Munich, Bayern, Germany

PIERRE BONVIN, Munich University of Applied Sciences, Munich, Bayern, Germany

Open Access Support provided by:

Siemens AG

Fortiss GmbH

Munich University of Applied Sciences

PDF Download
3639477.3639744.pdf
22 January 2026
Total Citations: 6
Total Downloads: 539

Published: 14 April 2024

Citation in BibTeX format

ICSE-SEIP '24: 46th International Conference on Software Engineering: Software Engineering in Practice
April 14 - 20, 2024
Lisbon, Portugal

Conference Sponsors:
SIGSOFT

ICSE-SEIP '24: Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice (April 2024)
https://doi.org/10.1145/3639477.3639744
ISBN: 9798400705014

2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)

Automated Security Findings Management: A Case Study in Industrial DevOps

Markus Voggenreiter
Siemens Technology, LMU Munich
Munich, Germany

Florian Angermeir
fortiss GmbH
Munich, Germany

Fabiola Moyón
Siemens Technology, Technical University of Munich
Munich, Germany

Ulrich Schöpp
fortiss GmbH
Munich, Germany

Pierre Bonvin
pierre-louis.bonvin@hm.edu
Munich University of Applied Sciences
Munich, Germany

ABSTRACT

In recent years, DevOps, the unification of development and operation workflows, has become a trend for the industrial software development lifecycle. Security activities turned into an essential field of application for DevOps principles as they are a fundamental part of secure software development in the industry. A common practice arising from this trend is the automation of security tests that analyze a software product from several perspectives. To effectively improve the security of the analyzed product, the identified security findings must be managed and looped back to the project team for stakeholders to take action. This management must cope with several challenges ranging from low data quality to a consistent prioritization of findings while following DevOps aims. To manage security findings with the same efficiency as other activities in DevOps projects, a methodology for the management of industrial security findings minding DevOps principles is essential.

In this paper, we propose a methodology for the management of security findings in industrial DevOps projects, summarizing our research in this domain and presenting the resulting artifact. As an instance of the methodology, we developed the Security Flama, a semantic knowledge base for the automated management of security findings. To analyze the impact of our methodology on industrial practice, we performed a case study on two DevOps projects of a multinational industrial enterprise. The results emphasize the importance of using such an automated methodology in industrial DevOps projects, confirm our approach's usefulness and positive impact on the studied projects, and identify the communication strategy as a crucial factor for usability in practice.

CCS CONCEPTS

• **Software and its engineering** → Software creation and management; • **Security and privacy** → **Software and application security**.

This work licensed under Creative Commons Attribution International 4.0 License.

ICSE-SEIP '24, April 14–20, 2024, Lisbon, Portugal
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0501-4/24/04.
https://doi.org/10.1145/3639477.3639744

KEYWORDS

DevOps, security, software engineering, security findings management

ACM Reference Format:
Markus Voggenreiter, Florian Angermeir, Fabiola Moyón, Ulrich Schöpp, and Pierre Bonvin. 2024. Automated Security Findings Management: A Case Study in Industrial DevOps. In *46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '24), April 14–20, 2024, Lisbon, Portugal.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3639477.3639744

1 INTRODUCTION

Over the last few years, the software development strategy in the industry has changed from traditional, waterfall-oriented models to iterative and incremental models. A change in the distribution, operation, and maintenance of industry software further caused many practitioners to follow DevOps principles [8]. Amongst multidisciplinary collaboration in the project, one of the key advantages is the automation of tests [26]. To comply with industrial security standards and best practices [2, 11, 18, 22, 30], integrating security aspects in the automation testing strategy is relevant for industrial companies [17], and represents a commonly recommended measure in software development lifecycles [14]. Due to the complexity and size of industrial software, the aspects under test range from software and infrastructure code over configurations and third-party dependencies to the running applications and their production environment.

In practice, problems with the security of the software can be identified in each of its components. Hence, each security test category adds another perspective on the overall picture of the software security status and consequently utilizes a different terminology to describe the flaw. Regardless of the security problems cause, it must be managed and reacted to. For the purposes of this manuscript, we define the term *security finding* as any weakness related to the security of a software product that was identified but not yet confirmed or further processed. This definition is aligned with the ISO security standards treating software vulnerability analysis, which define weakness as software product characteristics that, in proper conditions, could contribute to introducing vulnerabilities [12]. Such security findings are detected, e.g., during automated

security tests in CI/CD pipelines, manual code reviews, or continuous monitoring activities in production environments. Examples range from hard-coded passwords identified early in the life cycle to publicly known vulnerabilities found during monitoring.

Minding DevOps principles, security findings must be managed in close collaboration between stakeholders, security experts, and the development team with the ability to mitigate or fix them while trying to reduce the size of work packages and minimize lead time. However, the automation of tests often results in huge low-quality data sets, with security-specific terminology acquired at distinct stages of the software engineering life cycle. Dealing with them manually requires substantial time and effort from the entire project team. Consequently, the management of security findings in the industry represents a considerable challenge, especially if it should be performed with the same efficiency as other DevOps practices.

This paper proposes a methodology for the management of security findings in industrial DevOps projects. We evaluate the impact of the methodology on industrial practice by conducting a case study with two software engineering projects. In this context, we developed an instance of the methodology, called the **Security** **F**eedback **L**oop **A**nalysis and **M**anagement **A**pplication (*Security Flama*). The *Security Flama* collects security testing reports from various sources and automatically improves data quality, supporting practitioners to track and take action on security findings. In addition, it communicates evidence-based results tailored toward the individual needs of different stakeholders, such as developers, project managers, or security experts. The evaluation provides three main outcomes. First, the ability to continuously visualize the current state of security findings has shown to be beneficial for industrial software projects. Second, our methodology in particular was perceived as highly useful and has noticeably impacted the amount of security findings in one of the projects. Finally, the communication strategy with the project team and stakeholders was identified as a key factor in looping back security findings information from testing to initial DevOps phases like plan and code. In summary, both the methodology and its instance, the *Security Flama* implement the *security feedback loop*, a concept we define as applying the DevOps principle of continuous feedback [15] for the use case of security findings in the software product.

Contributions With this paper, we make the following contributions: 1. A methodology for the automated management of security findings in industrial software engineering projects following DevOps principles. 2. Guidance on how to instantiate the methodology, including its key features, in order to replicate the *Security Flama*[1] for industrial usage. 3. An industry case study on the potential of the methodology gathering quantitative and qualitative data simultaneously on the management of security findings.

2 BACKGROUND AND RELATED WORK

To foster productive collaboration between team members with varying backgrounds, organizations turned towards DevOps to bridge the gap between development and operations [8, 24] using cross-functional teams [7, 41]. However, neither academia nor industry has a universal definition for DevOps and the principles that should be followed [7]. Humble and Molesky propose four core principles, when bridging the gap between development and operations [9]. Similarly, Gene Kim describes the fundamental principles of DevOps as the "three ways" [15], covering parts of the concepts provided by Humble. Moreover, *ISO/IEC/IEEE 32675* specifies the practices for operations teams, development teams and other stakeholders on collaboration for a successful building and deployment of systems [13]. In summary, key elements include a culture of collaboration, automation of repetitive tasks, and continuous measurement. Particularly the automation of repetitive security testing and the resulting security reports motivates our research on security findings management.

Dealing with security problems in software development projects is, however, not a new research field. Specifically, the field of vulnerability management as part of risk management is well researched with proposed solutions ranging from enterprise- to project-level [20, 34, 40]. With concepts like unique vulnerability identifiers (CVE-IDs [3]) and vulnerability severity ratings (CVSS [10]) this domain contributed widely employed processes and technologies that are crucial for industrial practitioners. For example, Farris et al. propose a framework for the management of vulnerabilities with a focus on the prioritization of vulnerability mitigation [5]. However, vulnerability management mostly focuses on problems existing in productive systems on the operations side of the software engineering lifecycle. In the best-case scenario, security problems never reach production and are instead mitigated in earlier stages, following the fast fail and shift-left mindset of DevOps.

Towards a more holistic view, Rindell et al. propose to consider any deviation from the intended state of security as technical debt. This includes missing security features in the software and security bugs/defects alike [25]. With the trend towards DevOps and continuous deployment, they saw the necessity for a continuous process that analyzes, validates, and keeps track of the security debt identified by tooling. An exemplary application for this use case was employed by Torkura et al. for viewing the security status history of service instances [35]. They utilized the vulnerability management system *DefectDojo* to collect security test results and visualize across multiple microservices. In contrast to other vulnerability management systems, *DefectDojo* handles various types of test reports accumulated throughout the entire lifecycle. However, its main goal also represents its biggest drawback: "The top goal of DefectDojo is to reduce the amount of time security professionals spend logging vulnerabilities" [1]. Hence, the audience is security professionals, neglecting the needs of stakeholders such as developers, or project managers to effectively collaborate on security findings management

However, DefectDojo is not the only tool supporting the management of security findings. Other representatives of this group include Faraday or Sonarqube. Faraday is a tool suite for the identification and management of security vulnerabilities in assets and networks [4]. Their approach focuses on the analysis of existing infrastructure and the management of findings reported by these scans, indicating that solely findings from the operations stages are minded. In a similar fashion, Sonarqube provides the capability to conduct static code analysis and manage resulting security findings [29] on their platform. The platform allows the upload of

[1]The reader may understand that the Security Flama source code is protected by a non-disclosure agreement with the industry partner.

third-party reports from other tools conducting static code analysis or linting. However, the management of findings originating from external sources lacks functions like finding validation, which are exclusively enabled for internally identified security findings. Moreover, Sonarqube focuses on software code, disregarding later stages of the software development lifecycle.

Another approach is the usage of issue trackers to manage security findings in practice [23]. However, security findings are not equal to most other type of bugs [42]. Even though they are mostly treated with a higher priority, they require more experienced developers to fix and are more frequently re-opened again. Moreover, the data quality of automated security testing results necessitates further investigations before they can even be considered as actual issues, minding shortcomings like False Positives. Therefore, the management of security findings has to start before any issue tracker can be utilized. Hence, we see a gap in the existing State-of-the-Art for security findings management in industrial DevOps projects.

3 MANAGING SECURITY FINDINGS IN INDUSTRY

The first step towards improving the security findings management process in industrial software engineering projects is the development of a comprehensive methodology supporting practitioners in their work. Following the approach of Design Science Research [27, 33], we conducted research within Siemens AG to identify the challenges in practice. Siemens is a multinational industrial enterprise with software development activities ranging from the healthcare sector to mobility. In advance to the development of the methodology, we interviewed practitioners at the industry partner supporting the secure development of industrial software engineering projects. During these unstructured interviews, we asked them about issues they experienced in DevOps projects following a traditional security findings management approach. The results of these interviews coincide with the challenges mentioned in the introduction, logically emerging from the interconnection between software development in a domain with high-security demand and modern DevOps principles. These challenges and the necessity to address them are identified by commonly accepted maturity models, frameworks, and standards [2, 11, 19, 22, 30]. Therefore, we consider these problems as widespread and their solution as a contribution to industry and academia alike.

Following the Design Science Research approach, a respective solution approach was designed and instantiated for each identified challenge. In the following, we present the challenges grouped by topic and their treatments considering DevOps principles as established in the methodology.

3.1 Data Quality Improvement

Challenges. The first area of challenges we identified was related to the data quality of the source data. With various tools employing different perspectives on the product, each security report has a **different data format**. Furthermore, each security activity might be located at a different project resource resulting in **distributed source locations**. Moreover, these tools might overlap in their coverage, introducing **duplicate security findings**. Since most security tools were created by security professionals, the **terminology of findings** it generates demands practitioners to have domain expertise to understand the results.

Treatments. To improve the quality and availability of the data delivered by security activities, automated preprocessing is crucial. Since each project utilizes different tools, a tool-agnostic processing must be in place. In our methodology, the data in the form of security reports is collected from each tool and made centrally available for subsequent processing stages. Next, the data format is unified by a parsing operation utilizing a mapping between each security activity format and a common data format. The fields of the data model, which can be found in the supplementary material [36], represent the data commonly found in security findings. To avoid duplicate security findings and provide an aggregated dataset to the later stages, our methodology follows the approach by Schneider et al. [31] for the clustering of security findings with semantic similarity-based techniques. This means, duplicate findings are identified by comparing textual similarities in their fields. Finally, each finding is investigated for opportunities to refine the comprised information and, therefore, simplify the comprehension of the finding's content for practitioners. This rule-based enrichment utilizes an if-then structure to identify applicable security findings that are subsequently enriched by the properties previously defined in the rule. Since the relevance of enrichment is highly project-specific, almost no general rules are given. The only default rule in our methodology is a tool-dependent explanation of how each finding was identified (E.g., *The tool <Tool> identified this finding, by querying each of your dependencies in public vulnerability databases. If a vulnerability is found for a component, it creates a finding.*). The solution approach for the *Data Quality Improvement* can be found on the left side of Figure 1.

This stage can be fully automated during the project. Solely the parser mapping each source activity and rules for enrichment must be defined upfront.

3.2 Finding Analysis Support

Challenges. To enable practitioners during their work on security findings, respective insights into the current state of security must be accumulated. One indicator of the relevance and prevalence of security findings is the identification history. This comprises the occurrence of findings in security reports, including factors like first or last identification. Moreover, each security finding reaches different states during its lifecycle that must be maintained consistent throughout continuously new data being reported by security activities. Finally, the treatment of each finding depends on multiple factors, including but not exclusively being the severity of security findings. Since the time available for changes to the software is limited by the size and maturity of the development team, prioritizing the response to findings is crucial.

Treatments. To support practitioners in analyzing and responding to security findings, three activities are aided by the methodology. First, the methodology tracks the history of each finding, covering all occurrences in security activity reports. This provides historical data on each finding, including the first and last identification, its frequency, and distribution across sources. Moreover, the

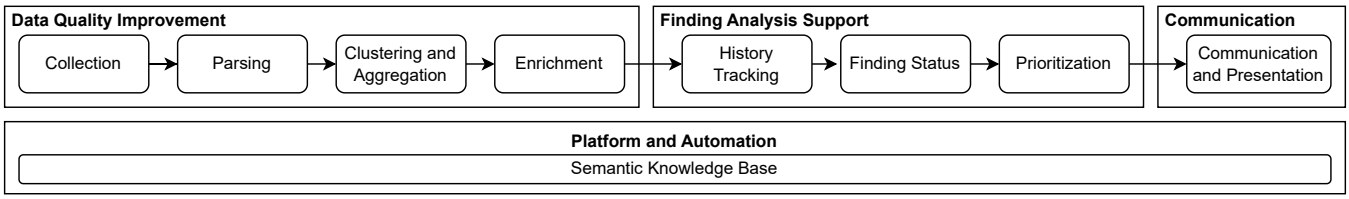

Figure 1: Methodology Design for the Security Findings Management in Industrial DevOps Projects

status of each finding is tracked. This status represents the result of a finding verification and persists throughout incremental finding changes. The available values for the finding status comprise: "Open", "In Work", "False Positive", "Invalid", "Accepted", "Solved", "On Hold" and "Disappeared". A finding's status persists either permanently or under certain circumstances (e.g. finding was not found in the last two reports). By default, every finding receives the status "Open", which is solely changed to "Disappeared" if the finding was not found in the last report. "Disappeared" is furthermore the only status that cannot be assigned by users of the methodology. Finally, each finding must be prioritized to cope with reduced available implementation time per iteration. Our methodology supports this prioritization process by providing a common finding severity score and a project-dependent prioritization score for each finding. We follow the approach by Voggenreiter and Schöpp [38] for the calculation of both scores. This implies that the severity of each finding is calculated by applying activity-based models to the findings data, which is afterward manually refined by the project team or stakeholders to present the importance of caring about this finding on a numeric scale. Finally, our methodology requires that all data resulting from the above-mentioned process steps is documented and traceable at any time. The solution approach for the *Finding Analysis Support* can be found in the middle of Figure 1.

3.3 Security Feedback Communication

Challenges. In order for practitioners to fulfill their tasks based on the security findings, knowledge about findings must be communicated to them. Since the subsequent actions depend on the role of the knowledge-receiving entity, the **communication must be tailored towards the user's role**. Minding the importance of cross-domain collaboration, all **stakeholders and team members require access to the security findings data**. However, the data must also be **accessible to automated processes**, e.g., to provide cross-project correlations. Finally, the **data must be continuously available**, so that the team can work on the management of security findings whenever time allows it.

Treatments. To communicate the security feedback, our methodology defines a common interface to practitioners and processes. For the processes, an automated interface in the form of an API is necessary, while human users require a visual approach using a webinterface. Both interfaces provide access to all security findings and their related information. Moreover, role-based views on the data provide users with the information necessary for their tasks. To contribute to a tailored communication strategy, a specific view for developers focusing on the solution aspects of the management is necessary.

This stage can be fully automated, except for additional role-based views that might be necessary. The solution approach for the *Security Feedback Communication* is depicted on the right side of Figure 1.

3.4 Platform and Automation

Challenges. With all previous solution approaches defined, performing them manually is unfeasible. Hence, all **steps must be orchestrated and automated** as far as possible. The information acquired during the methodology must be **centrally documented and available**, to access the information at any time. Moreover, our methodology solely presents a snapshot of generally existing challenges. Further process steps might be employed at more sophisticated projects, therefore requiring the platform to be **customizable** to support these as well.

Treatments. Finally, all preceding solution approaches must be orchestrated and automated within a common platform to comply with DevOps principles and avoid manual, repetitive work. For this platform, we follow the idea of using a semantic knowledge base as a platform for the management of security findings [37]. This knowledge base represents a data source for each project which maintains consistent information throughout new reports being added by sources or data being changed by external influences like developers. We extend their work by adding the concept of queries to the knowledge base, introducing temporarily computed views on the data that are customized according to time and necessary information. This results in the following four concepts:

- Belief: Any type of data ranging from security reports to the prioritization score of single findings
- Rules: Logic, describing how to derive new Belief from existing Belief
- Query: Computational Rules, which provide information based on the current elements of Belief in the knowledge base
- Data Storage: Underlying storage element, storing Belief, Rules, and Queries

Applied to our previous solution approaches, each piece of information is considered as a belief. Every data processing step is a rule transforming, e.g., a security report to multiple security findings by applying parsing rules.

Implementation-wise, the knowledge base uses Elasticsearch as *Data Storage. Belief* is stored as documents in the search engine while *Rules* are coded in Python. The *Queries* are Python snippets, including Elastic Query DSL requests to access belief in the knowledge base. The knowledge base is maintained consistent by a logical

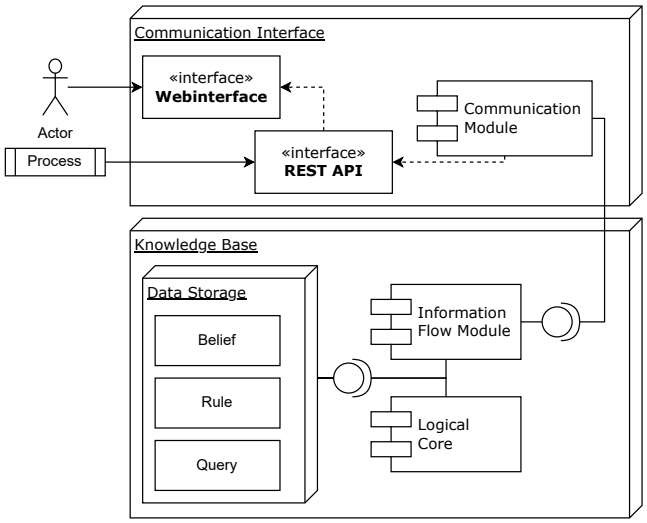

Figure 2: Components of the *Security Flama*

core that ensures that new belief is derived from existing belief and falsified data is corrected after new insights occur. Access to the data is provided by two interfaces. Automated processes can access the knowledge base via a RESTful API, while users can access a visual representation via a website. The website includes a customized, role-based dashboard, a sort- and filterable list of all findings, and a separate page for each finding. Examples for these pages can be found in the supplementary material [36]. We call this instantiation of the methodology the *Security Flama*. Its components are depicted in Figure 2.

3.5 Processing Example

A typical workflow for the *Security Flama* starts with new security reports being generated in a CI/CD pipeline with automated security tests. Using a separate job in the pipeline, the resulting reports are uploaded to the REST API and forwarded to the Information Flow Module of the *Security Flama* (see Figure 2). This module adds the new reports as instances of the belief class "Security Report" to the data storage and informs the logical core about a change in the dataset. The logical core checks, whether any rule is triggered by a new instance of this class and identifies the rule, which parses security reports to distinct findings. Executing the Python code of this rule, the logical core uses the new security report as input and returns new instances of the belief class "Security Finding" representing the parsed, distinct findings. The newly constructed security findings are added to the data storage again and the logical core re-evaluates whether any rule is triggered by new instances of security findings in the data storage. This process is iteratively executed, with the logical core analyzing whether any rule can be used to derive new information and the execution of applicable rules. In our example, this results in the subsequent execution of the deduplication and aggregation using Latent Semantic Indexing, rule-based enrichment, history and status tracking, and the modeling of severity and priority for each security finding. This results in additional instances of belief classes being added to the knowledge base. Afterward, the knowledge base reaches a consistent state, as no additional belief can be derived from the newly added security reports. This implies that the processing of the uploaded security reports concluded.

After the automated tests have been conducted, project team members might be interested in the currently existing security findings. Therefore, the next part of the workflow would be a user of the *Security Flama* accessing the webinterface, which again forwards the request to the Information Flow Module. This module computes the response to the request by accessing the knowledge base queries, which again query the belief instances, constructing answers to questions like "What are the ten findings with the highest priority" or "How many findings are currently open". This knowledge is presented by the webinterface based on the response of the Information Flow Module. Any potential user input, e.g., refinement on the priority of security findings, is also implemented via the communication interface and stored as belief. Finally, users would enter any high-priority security finding into their project-specific issue tracker or backlog to track its implementation.

4 EVALUATION

Using the methodology described in the last section, we collaborated with our industry partner once again to verify our methodology as treatment to the challenges in practice. In this section, we describe this evaluation by using the *Security Flama* as an instance of it.

4.1 Evaluation Planning

The close relation of our methodology to the effectiveness of the software development lifecycle necessitates an evaluation covering quantitative performance indicators of a real-world project as well as qualitative perception data by users of the methodology. Consequently, we evaluate the *Security Flama* through the following research questions (RQ):

- **RQ1: How does the usage of the *Security Flama* impact the development process indicators?**
- **RQ2: Is the *Security Flama* perceived as useful by the project team for managing security findings?**
- **RQ3: What are the benefits and limitations of applying the *Security Flama* for project stakeholders?**

Towards this goal, we conduct a case study, following the State-of-the-Art for empirical software engineering and case study research [6, 16, 28, 32] minding challenges and domain experience [21].

As subjects, we require ongoing software development projects at our industry partner. Each project has to follow DevOps principles, run for more than one year, utilize automated security tests with a subsequent security findings management process, and contain weekly code changes. Our partner provided us with two independent projects in the R&D domain fulfilling our demands, consisting of two (Project A) and three developers (Project B), respectively. Both projects employed a Kanban board for task planning and the issue tracker integrated into the code repository to manage coding-related problems. Their existing security findings management process comprises the manual review of security testing reports in the CI environment and a subsequent discussion of these results

Table 1: Quantitative Data Collected

Indicator
Number of security activities producing findings in the project.
Number of security tools providing data to the *Security Flama*.
Number of reports created per week.
Number of raw findings identified per week.
Number of aggregated findings per week.
Number of new findings per week.
Number of findings with status X (Open, False Positive, ...).
Number of findings with severity X (Critical, High, ...)
Number of user input per week (Status, Prioritization).

in the weekly project team meeting, potentially resulting in a new issue in their backlog. Since the management of security findings is up to each project, this strategy is not representative of all projects at our partner. Projects utilize a variety of combinations between different code repositories, issue trackers, and task planners.

Each project follows a rigorous protocol during the evaluation:

(1) Integration of the *Security Flama*
(2) Passive, continuous collection of statistics from the project
(3) Introduction of the *Security Flama* to the project team
(4) Data collection with five iterations of questionnaires
(5) Conclusion of data collection with interview
(6) Ramp-Down

Initially, the *Security Flama* is integrated into the ongoing projects and collects security reports resulting from security tests of the main branch with no introduction of the methodology to the project teams. In the first four weeks after integration, the projects are passively monitored without intervention to achieve a baseline of data. This monitoring is implemented by a weekly collection of the quantitative data listed in Table 1. Afterward, the methodology is presented to the project team, including developers and other stakeholders, and access to the user interface is established for all users. Furthermore, the case study and its goals are presented to the subjects. To capture potential changes in the perception of the *Security Flama* over time [39], each team member is interviewed bi-weekly, focusing on the experience of the past two weeks. The interview consists of a questionnaire comprised of open and binary questions as well as questions on a Likert Scale defined from 1 - "Strong Disagree" to 6 - "Strong Agree", avoiding any neutral answer. After five iterations of the reoccurring interviews, the evaluation is concluded by a final interview. This interview assesses the subjective *usability*, *benefits*, and *limitations* of the *Security Flama* utilizing a distinct questionnaire. The initial presentation, both questionnaires and the interview guide can be found in the supplementary material [36].

4.2 Evaluation Results

The results presented in this section are published in accordance with the collaboration agreement with our industry partner. Consequently, the results are partially anonymous to comply with this agreement.

Quantitative Results:

Over the course of the evaluation, we collected the quantitative evidence on 16 occasions. In both projects all security activities were automated, implying that the amount of security activities equals the amount of security tools. In Project A, a secret scanning tool, a static code analysis, and a third-party vulnerability testing tool produced the security reports managed by the *Security Flama*. In Project B, an additional, tailored third-party vulnerability testing tool was added on February 15 during the evaluation period, after a gap in the testing coverage was identified during the review of security findings.

The aggregated data collected in both projects can be found in Table 2. The results of Project A are depicted on the left of each column, while the results of Project B are shown on the right. The statistics calculated about the analyzed indicators can be found in Table 3. We consider a "New Finding" to be first identified in the last seven days, hence since the last data point. We excluded the first date from this calculation, as initially all findings are new. All values are rounded to the nearest lower integer, except for the "User Input per Week". The "Amount of findings with severity X / status X" is excluded from the table due to its verbosity. Instead, the severity of findings over time are presented in Figure 3 for Project A and in Figure 4 for Project B. Automated changes towards the finding status are presented in Figure 5 for Project A and in Figure 6 for Project B. The only manual change in finding status appeared in Project A in the week of March 20. Two findings were assigned the status "False Positive" and "Accepted". Moreover, the number of reports, raw findings, and aggregated findings of both projects can be found in Figure 7.

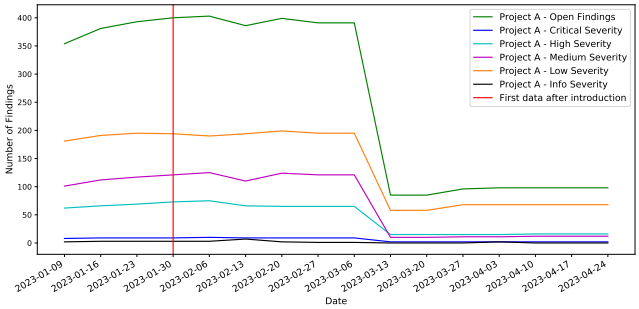

Figure 3: Severity of Findings in Project A

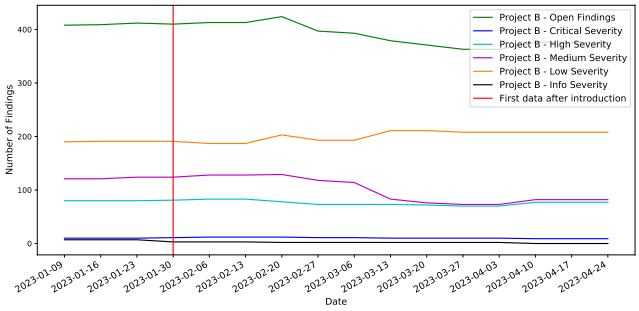

Figure 4: Severity of Findings in Project B

Table 2: Quantitative Data of Projects A and B

date	number reports		number findings		parsed findings		new findings		open findings		number prio		sever. crit		sever. high		sever. medium		sever. low		sever. info		status Open		status Disappeared		status FP		status Accep.	
2023-01-09	214	95	1087	1112	354	408	354	408	354	408	0	0	8	10	62	80	101	121	181	190	2	7	1087	1112	0	0	0	0	0	0
2023-01-16	672	307	1126	1143	381	409	27	1	381	409	0	0	9	10	66	80	112	121	191	191	3	7	1126	1143	0	0	0	0	0	0
2023-01-23	790	425	1142	1156	393	412	12	3	393	412	0	0	9	10	69	80	117	124	195	191	3	7	1142	1156	0	0	0	0	0	0
2023-01-30	1192	1266	1193	1184	421	427	28	15	400	410	0	0	9	11	73	81	121	124	194	191	3	3	1153	1155	40	29	0	0	0	0
2023-02-06	1595	1692	1226	1217	436	442	15	15	403	413	0	0	10	12	75	83	125	128	190	187	3	3	1151	1153	75	64	0	0	0	0
2023-02-13	1985	1692	1480	1217	472	442	36	0	386	413	0	0	9	12	66	83	110	128	194	187	7	3	1124	1153	356	64	0	0	0	0
2023-02-20	3874	2496	1596	1574	508	502	36	60	399	424	0	0	9	12	65	78	124	129	199	203	2	2	1179	1236	417	338	0	0	0	0
2023-02-27	4360	3316	1612	1614	514	515	6	13	391	397	0	1	9	11	65	73	121	118	195	193	1	2	1145	1142	467	472	0	0	0	0
2023-03-06	4406	3712	1620	1643	514	518	0	3	391	393	0	0	9	11	65	73	121	114	195	193	1	2	1146	1128	474	515	0	0	0	0
2023-03-13	4582	3856	1638	1767	523	527	9	9	85	379	0	0	2	10	15	73	10	83	58	211	0	2	198	1085	1440	682	0	0	0	0
2023-03-20	4582	4882	1638	2034	523	588	0	61	85	371	0	0	2	10	15	72	10	76	58	211	0	2	198	1068	1440	966	0	0	0	0
2023-03-27	4825	5150	1652	2210	534	615	11	27	96	363	0	1	2	10	15	70	11	73	68	208	0	2	210	1075	1440	1135	1	0	1	0
2023-04-03	5409	5309	1658	2210	537	615	3	0	98	363	0	1	2	10	15	70	11	73	68	208	2	2	214	1075	1442	1135	1	0	1	0
2023-04-10	5508	6082	1670	2290	546	651	9	36	98	376	0	1	2	9	16	77	12	82	68	208	0	0	214	1100	1454	1190	1	0	1	0
2023-04-17	5508	6082	1670	2290	546	651	0	0	98	376	0	1	2	9	16	77	12	82	68	208	0	0	214	1100	1454	1190	1	0	1	0
2023-04-24	5508	6082	1670	2290	546	651	0	0	98	376	0	1	2	9	16	77	12	82	68	208	0	0	214	1100	1454	1190	1	0	1	0

Table 3: Quantitative Statistics per Week

Indicator	Project A			Project B		
	Min	Max	Avg	Min	Max	Avg
#Security Activities	3	3	3	3	4	4
#Security Tools	3	3	3	3	4	4
#Reports	0	1889	344	0	1026	380
#Raw Findings	0	1087	104	0	1112	143
#Aggregated Findings	0	354	34	0	408	40
#New Findings	0	36	12	0	61	17
#User Input	0	2	0.25	0	1	0.125

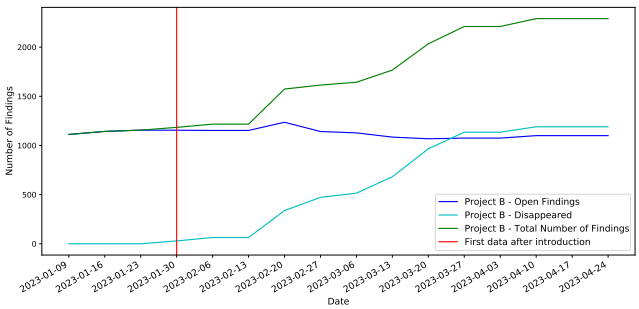

Figure 6: Findings Status of Project B

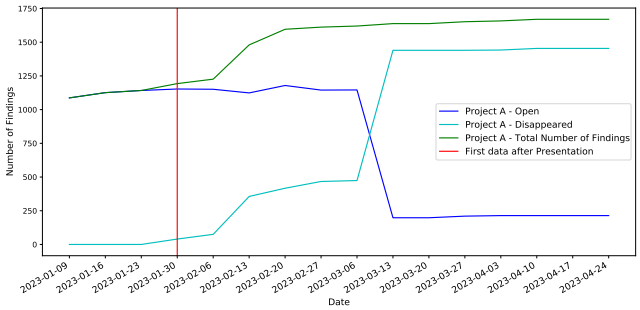

Figure 5: Findings Status of Project A

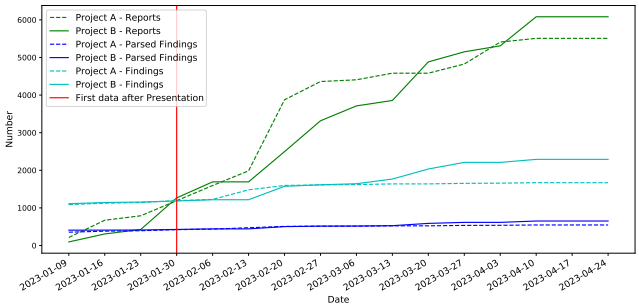

Figure 7: Finding Statistics of both Projects

Qualitative Results - Repeated Interviews:
In addition to the quantitative data, the reoccurring interviews with the project teams supplied our evaluation with qualitative information about the performance of the *Security Flama*. In summary, each team member was interviewed five times based on the reoccurring questionnaire and one time with the final questionnaire between February 13 and April 28. One developer left Project B in March, resulting in the completion of just two of five interviews for this subject. Especially during the first interview sessions, the subjects were hesitant to answer questions, as they felt unable to do so with the limited time they spent on using the *Security Flama*. Most answers on a Likert scale were answered positively with a "5-Agree" or "6-Strong Agree". Only questions 17 and 12 were exceptions. Question 17, asking whether the subject was restricted in its work by the *Security Flama* in the last two weeks, was answered with "1-Strong Disagree" consistently. Another exception was question 12, asking about the transparency of how the *Security Flama* works. This question showed a development over time, depicted in Figure 8. The answers to this question ranged from "1-Strong Disagree" to "5-Agree".

Since answers to the open questions contained confidential data on active vulnerabilities, these are abstracted for presentation. The subjects mentioned the overview of the currently existing security findings in the product as notably useful and the main benefit of

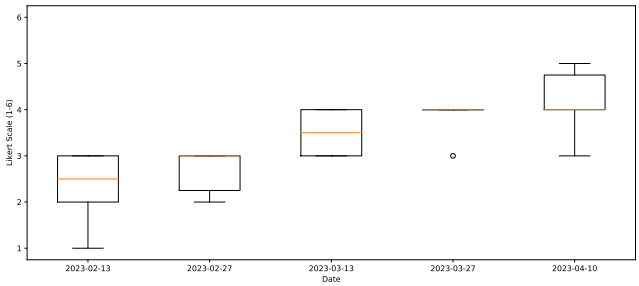

Figure 8: Perception on the Transparency of the *Security Flama* over Time

the *Security Flama*. The referenced aspects ranged from directly identifying new findings, over having an aggregated view on the findings list to the common severity scale allowing to compare findings with each other. However, all subjects mentioned these aspects in the context of gaining "overview". According to the interview results, the *Security Flama* itself was mainly used in preparation for or during team meetings to align the response to security findings in the team. The only limitations mentioned during the interviews were bugs in the implementation of the webinterface. Similarly, the *Security Flama* improvements and general comments of subjects exclusively addressed potential changes in the webinterface implementation.

Qualitative Results - Final Interviews:
To conclude the evaluation, six final interviews were conducted with the remaining team members. The results of these interviews are separated into the three assessed properties of the *Security Flama*. Its *usability*, the *benefits* of using it, and its *limitations* in the project. One subject felt uncomfortable answering the interview questions due to their insufficient direct interaction with the *Security Flama*, resulting in five interviews.

All subjects confirmed the usability of the *Security Flama*, describing it as "highly beneficial" or "extremely useful". The collectively mentioned reason is the "up-to-date insight on the security state" of a product, covering "newly introduced findings" and allowing for "insights otherwise impossible". The second most mentioned factor in usability is the simplicity of accessing the security findings data.

As benefits of the *Security Flama* our subjects mentioned various aspects. The most commonly mentioned one, was its flexibility to tailor the tool to the needs of the project in terms of security activities used and user input. The evidence-based overview on the security state of the software was again mentioned by all subjects as a benefit. Moreover, some components of the *Security Flama* were mentioned as beneficial, including the common datamodel, the aggregation of findings, the calculation of severity on a common scale, and the "role-tailored communication of data".

Limitations of the *Security Flama* mentioned by the subjects can be categorized into three clusters. The first cluster addresses the limitations imposed on the project when using the *Security Flama*. The subjects described the effort to actively manage findings as "exhaustive" and often forgot about it. One subject wished for a more proactive communication approach like email notifications if critical findings occur, which the team could subsequently react to. Moreover, this team also managed every finding that should be fixed in the backlog. Thus, findings were prioritized against other backlog items, resulting in the subject's perception of "duplicate housekeeping". Consequently, the manual effort to interact with the *Security Flama* was seen as a limitation for the project. The second cluster addressed shortcomings of the *Security Flama* and potential procedural improvements. The subjects mentioned the poor visibility of how the data is processed by the *Security Flama* and the complexity of adding priority scores with numbers as shortcomings. Moreover, they perceived the lack of a long-term success overview for all roles to be detrimental to the team's motivation. One subject observed the necessity for a change in the finding status system. According to the subject, the finding status was not adapted in their project, as team members wanted to confirm a finding fix by checking whether it disappeared. Finally, multiple subjects gave recommendations on how to improve the implementation of the webinterface. However, these were not related to the *Security Flama* directly, but just related to aspects of the communication.

5 DISCUSSION

In this section, we discuss the results of our evaluation, starting with general discoveries and focusing on answering the research questions in a second step. Finally, we address threats to the validity of our research.

5.1 General Analysis of Evaluation Results

The data acquired during the evaluation shows multiple features, that must be discussed upfront before addressing the research questions.

First, we have to correct our initial assumption that all team members and stakeholders interact with the methodology directly. Contrary to our initial assumption, one subject accessed the data exclusively through other members during team meetings or on specific occasions. These subjects, either missing the time or interest to work with security findings, represent a new type of methodology user that has not been considered yet.

To interpret the finding status, it is important to note that each location of a finding in the product has a distinct status, and numerous locations can be aggregated within the same finding. Consequently, the sum of all status values is equal to the number of raw findings presented in "number findings". Since each aggregated finding is assigned a severity, the sum of all severity levels is equal to the number of aggregated findings presented in "parsed findings".

Moreover, an increase in security findings can be observed for Project B on April 20 (see Figure 6). This is explained by the additional security tool being added on February 15.

As seen in Table 2, the data on the "2023-04-10", "2023-04-17", and "2023-04-24" shows the exact same values. This is due to the public Easter Holidays, resulting in no code changes within both projects. These missing data changes can also be observed, whenever code changes and tests only happened on project branches since only the results affecting the main branch were used for findings management.

Finally, it is crucial to mention that the number of reports, raw and aggregated findings can only increase over time as depicted in

Figure 7. This is due to the methodology's history tracking, which persistently stores all external data and the belief derived from it.

5.2 Answers to Research Questions

Given the data presented in the last section, we answer the research questions in this subsection.

RQ1: How does the usage of the *Security Flama* impact the development process indicators? The impact of the *Security Flama* can be visually recognized in Figure 5 and Figure 6. First, we observe that the aggregation diminishes the number of findings by at least two-thirds in both projects (see Figure 7). Moreover, in the week of March 6, one project team responded to multiple security findings based on the insights provided by the *Security Flama*, which drastically reduced the number of findings in the project. According to the interviews, this was possible due to the unique insights provided by the *Security Flama*.

While both projects show a reduction of open, unsolved findings over the course of the evaluation (green line in Figures 3, 4), this decrease can only be considered as substantial in Project A. Therefore, we conclude that the success of the *Security Flama*, as any security tool, highly depends on its acceptance and the importance of software security to the stakeholders. Also, the distribution of finding severity shown in Figure 3 and Figure 4 within both projects did not change notably.

> One project showed a substantial improvement in the process indicators when using the *Security Flama*. The impact on the other project was marginal due to the lack of adoption.

RQ2: Is the *Security Flama* perceived as useful by the project team for managing security findings? During the interviews, all subjects described the *Security Flama* as useful for the project and their work. Especially in Project A, the subjects argued that the *Security Flama* enabled them to identify quick wins for the reduction of the overall number of findings (see Figure 5). This considerable impact on the security of the software was traced to evidence-based visibility provided by the *Security Flama* in the reoccurring and final interviews.

Even though the subjects claim that the *Security Flama* was perceived as useful, we see some restrictions when analyzing the quantitative data. The overall user input provided to the *Security Flama* is remarkably low. As seen in Table 2, only one finding was assigned a priority score and two findings received a manually assigned status. The interview data gives various explanations for this behavior. Our subjects explain the absence of priority data with (1) the high complexity of using numeric scores as user input and (2) the approach of both projects in which only high and critical findings can and will be addressed at their current development stage. Consequently, a separate, more granular prioritization of findings was unnecessary in both projects. The low assignment of a finding status, on the other hand, was blamed on missing comfort functions in the web interface and the need for the automated confirmation of a finding not being reported again. Due to the binary decision about which findings are even considered relevant, one project wished for a mass tagging of all findings with severity medium and beneath as "accepted" by default. This would have affected 325 aggregated findings and therefore added more than 1000 status changes to the dataset of Project A at that time (2023-02-20).

We conclude that even though the *Security Flama* was perceived as highly useful, this perception seems to rely uniquely on the aspects that were also utilized. We assume that the prioritization would have been not perceived as useful if its usage was forced as part of the process. However, the ability to decide which aspects of the methodology are relevant for the project at its current state ensured its usefulness to the projects.

> Even though the *Security Flama* is perceived as highly useful, we believe that this perception relies on the ability to avoid interaction with inconvenient aspects of the methodology like manual prioritization.

RQ3: What are the benefits and limitations of applying the *Security Flama* for project stakeholders? The benefits and limitations of the *Security Flama* have been shown to be the most insightful part of the interviews.

Most subjects mentioned bugs in the web interface as limitations or shortcomings of the *Security Flama*. This shows the importance of a high-quality communication strategy, as it represents the single point of contact for practitioners to interact with the *Security Flama*. The communication strategy we followed of using a single web interface was described as insufficient during the final interview. One subject wished for a complementary proactive strategy with emails being sent, whenever new or critical findings arise as reminders to access the web interface. Moreover, integration with the product backlog was missing according to the subjects, resulting in duplicate housekeeping of security findings. Due to the broad variety of backlogs and issue trackers existing in practice, this was not implemented in the initial version of the *Security Flama*. Consequently, the implementation of our communication strategy needs to be further improved to better support the needs of practitioners.

However, not every limitation focused on the communication strategy. The current approach for prioritizing security findings was considered as not relevant for one project as prioritization happens in the respective backlog against other non-functional and functional requirements. Subjects of the other project perceived the user input necessary for prioritization as too complex. Even though we acknowledge the necessity of reducing the input complexity, we still believe that prioritization is crucial in projects that do not follow a strict policy of resolving only high and critical-severity findings.

Furthermore, the *Security Flama* has shown to be less transparent to its users than desirable. Especially during the first interviews, our subjects disagreed with the statement that the methodology was transparent in how data is processed. As shown in Figure 8, this improved over time so that the average opinion of subjects was a slight agreement with the statement during the last interview. This is, however, not acceptable as an unclear functionality could lead to acceptance issues in the team. A subject in the final interview further requested a processing view of the data, depicting the impact of each processing stage.

Even though our evaluation identified different limitations of the *Security Flama*, also several benefits were collected. The key benefit identified by our subjects was the evidence-based visibility over the

current level of known security findings in the software product. The perceived benefits of this continuous security overview covered several perspectives of the data (e.g., new findings, most critical findings), reinforcing the importance of knowledge base queries, as defined in Subsection 3.4. Furthermore, they described this view as particularly useful for discussing security findings in team meetings. Hence, we believe that the enablement of team collaboration on security findings represents the ultimate benefit of our *Security Flama*.

Another benefit we derive from the evaluation is that it supported all process steps of the security findings management in the projects. Even though all subjects agreed with the statement that the *Security Flama* covers all aspects of the security findings management in general, we believe that this claim cannot be generalized. In each project, different aspects of the security findings management are relevant. This creates a gap between claiming it was covering all aspects of the affected projects and covering all aspects in general. Therefore, we reduce the claim by concluding it was at least supporting all aspects of the selected projects.

Benefits: Evidence-Based Overview, Collaboration Enablement, Support for all Management Activities
Limitations: Communication Strategy, Transparency of Methodology, Effort for Userinput

5.3 Threats to Validity

As with any evaluation of research results, also the validity of our conclusions is affected by several threats.

First, the evaluation was conducted with the *Security Flama*, an instance of the methodology. This presents a threat to the internal validity of the research, as problems in the implementation could be attributed to limitations of the methodology. Since automation, however, is the central part of the methodology, it is impossible to verify the impact without instantiating it.

Another threat to the internal validity is the experience of our subjects with the management of security findings. During the evaluation, it was clear that some subjects were inexperienced in dealing with security findings and consequently associated advantages of the source data (suggested solution approaches) as benefits of the *Security Flama*. Conversely, data of low semantic value was attributed as a limitation of the *Security Flama*. Moreover, this also affects the construct validity of the interview data, as subjects with varying security expertise could interpret the questions differently. However, this represents practice realistically, as not every team member has expertise and experience in the security domain.

The environment of our evaluation represents the threat to the external validity of our results. Since both projects are conducted in the same company, multiple factors affect whether the results can be transferred to other companies and projects. Especially the size of the project teams and the investigation of just two projects impact the validity of the results. However, we accept this threat since it is common with evaluations in industrial practice and consider a large scale evaluation as part of our future work. Moreover, we encourage researchers and industry practitioners to replicate our work with potentially different perspectives. For that purpose, we provide the introductory material for the evaluation, the interview guide, and both questionnaires in the supplementary material [36].

5.4 Conclusion and Future Work

Managing security findings with the same efficiency as other DevOps practices challenges practitioners throughout the entire software engineering life cycle. In this paper, we proposed a methodology for the management of security findings aligned with DevOps principles. To evaluate the impact on industrial practice, we created the *Security Flama*, implementing our methodology as a semantic knowledge base for the management of security findings. The *Security Flama* was integrated into two ongoing software development projects at a multinational industrial partner and its impact was evaluated with quantitative and qualitative methods. We conclude that both the methodology and its instance, the *Security Flama*, establish the DevOps principle of continuous feedback [15] for security findings in software products while reinforcing other DevOps practices like cross-functional collaboration.

Our research yielded three key results. First, the evaluation within the context of two industry projects shows that the usage of an automated methodology for the management of security findings is crucial for industrial DevOps projects. Second, the proposed methodology for security findings management is beneficial for the investigated projects. This was evidenced by interviews with the team members as well as represented by the impact on the project performance indicators. However, it was also noticed that we initially considered some aspects of the methodology to be more beneficial as it has proven to be (prioritization). Third, the interview results reinforced the importance of a high-quality communication strategy and web interface. Most interview comments addressed limitations and improvements to the communication strategy, indicating its importance to our subjects.

As part of the last result, the evaluation subjects provided multiple suggestions on how to improve the methodology. A redesign of the communication strategy therefore sets our future work on the methodology, including e.g. a reduction of effort to add user input and a deep integration of project backlogs. Since both projects wished to continue using the *Security Flama*, the so-acquired long-term data will further complement the results of this paper in the future.

ACKNOWLEDGMENTS

All authors would like to thank the members of both projects for the active collaboration on the topic and for the valuable feedback. Furthermore, we would like to thank Berk Sudan and Sandip Sah for supporting the implementation of the Security Flama.

REFERENCES

[1] 2023. DefectDojo. https://owasp.org/www-project-defectdojo/
[2] BSIMM. 2021. Building security in maturity model. https://www.bsimm.com/
[3] MITRE Corporation. 2023. Common Vulnerabilities and Exposures. https://www.cve.org/
[4] Faradaysec. 2023. Faraday Security. https://faradaysec.com/
[5] Katheryn A. Farris, Ankit Shah, George Cybenko, Rajesh Ganesan, and Sushil Jajodia. 2018. VULCON: A System for Vulnerability Prioritization, Mitigation, and Management. *ACM Trans. Priv. Secur.* 21, 4, Article 16 (jun 2018), 28 pages. https://doi.org/10.1145/3196884
[6] Daniel Méndez Fernández and Jan-Hendrik Passoth. 2019. Empirical software engineering: From discipline to interdiscipline. *Journal of Systems and Software* 148 (feb 2019), 170–179.
[7] Brian Fitzgerald and Klaas-Jan Stol. 2017. Continuous software engineering: A roadmap and agenda. *Journal of Systems and Software* 123 (2017), 176–189. https://doi.org/10.1016/j.jss.2015.06.063

[8] Wilhelm Hasselbring, Sören Henning, Björn Latte, Armin Möbius, Thomas Richter, Stefani Schalk, and Maik Wojcieszak. 2019. Industrial DevOps. In *2019 IEEE International Conference on Software Architecture Companion (ICSA-C)*. 123–126. https://doi.org/10.1109/ICSA-C.2019.00029

[9] Jez Humble and J. Molesky. 2011. Why enterprises must adopt devops to enable continuous delivery. 24 (08 2011), 6–12.

[10] FIRST.Org Inc. 2023. Common vulnerability scoring system SIG. https://www.first.org/cvss/

[11] International Electrotechnical Commission. 2021. *IEC 62443 series*. International Electrotechnical Commission, Geneva, Switzerland.

[12] International Organization for Standardization. 2015. *Information technology — Security techniques — Refining software vulnerability analysis under ISO/IEC 15408 and ISO/IEC 18045* (ISO/IEC TR 20004:2015 ed.). International Organization for Standardization, Vernier, Geneva, Switzerland. https://www.iso.org/obp/ui#iso:std:iso-iec:tr:20004:ed-2:v1:en

[13] International Organization for Standardization. 2022. *Information technology — DevOps — Building reliable and secure systems including application build, package and deployment* (ISO/IEC/IEEE 32675:2022 ed.). International Organization for Standardization, Vernier, Geneva, Switzerland. https://www.iso.org/standard/83670.html

[14] Rafiq Ahmad Khan, Siffat Ullah Khan, Habib Ullah Khan, and Muhammad Ilyas. 2021. Systematic Mapping Study on Security Approaches in Secure Software Engineering. *IEEE Access* 9 (2021), 19139–19160. https://doi.org/10.1109/ACCESS.2021.3052311

[15] Gene Kim. 2018. *Phoenix project: A novel about it, devops, and helping your business win* (5 ed.). IT Revolution Press, Portland, OR.

[16] Santiago Matalonga, René Nöel, Gilberto Pedraza-Garcia, Hernán Astudillo, and Eduardo Fernández. 2017. *Generating Software Security Knowledge Through Empirical Methods*. https://doi.org/10.1201/9781315154855-4

[17] Fabiola Moyón, Rafael Soares, Maria Pinto-Albuquerque, Daniel Mendez, and Kristian Beckers. 2020. Integration of Security Standards in DevOps Pipelines: An Industry Case Study. In *Product-Focused Software Process Improvement*, Maurizio Morisio, Marco Torchiano, and Andreas Jedlitschka (Eds.). Springer International Publishing, Cham, 434–452.

[18] National Institute of Standards and Technology. [n. d.]. *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. National Institute of Standards and Technology, Gaithersburg, Maryland, United States. https://csrc.nist.gov/publications/detail/sp/800-218/final

[19] National Institute of Standards and Technology. 2022. *NIST Special Publication (SP) 800 series*. National Institute of Standards and Technology, Gaithersburg, Maryland, United States. https://csrc.nist.gov/publications/sp800

[20] Matunda Nyanchama. 2005. Enterprise Vulnerability Management and Its Role in Information Security Management. *Information Systems Security* 14, 3 (2005), 29–56. https://doi.org/10.1201/1086.1065898X/45390.14.3.20050701/89149.6

[21] Lotfi ben Othmane, Martin Gilje Jaatun, and Edgar Weippl. 2017. *Empirical research for software security*. Taylor & Francis Inc.

[22] OWASP. 2020. OWASP SAMM. https://owaspsamm.org/

[23] Nitish Pandey, Abir Hudait, Debarshi Kumar Sanyal, and Amitava Sen. 2018. Automated Classification of Issue Reports from a Software Issue Tracker. In *Progress in Intelligent Computing Techniques: Theory, Practice, and Applications*, Pankaj Kumar Sa, Manmath Narayan Sahoo, M. Murugappan, Yulei Wu, and Banshidhar Majhi (Eds.). Springer Singapore, Singapore, 423–430.

[24] Olivia H. Plant, Jos van Hillegersberg, and Adina Aldea. 2021. How DevOps capabilities leverage firm competitive advantage: A systematic review of empirical evidence. In *2021 IEEE 23rd Conference on Business Informatics (CBI)*, Vol. 01. 141–150. https://doi.org/10.1109/CBI52690.2021.00025

[25] Kalle Rindell, Karin Bernsmed, and Martin Gilje Jaatun. 2019. Managing Security in Software: Or: How I Learned to Stop Worrying and Manage the Security Technical Debt. In *Proceedings of the 14th International Conference on Availability, Reliability and Security* (Canterbury, CA, United Kingdom) *(ARES '19)*. Association for Computing Machinery, New York, NY, USA, Article 60, 8 pages. https://doi.org/10.1145/3339252.3340338

[26] Leah Riungu-Kalliosaari, Simo Mäkinen, Lucy Ellen Lwakatare, Juha Tiihonen, and Tomi Männistö. 2016. DevOps Adoption Benefits and Challenges in Practice: A Case Study. In *Product-Focused Software Process Improvement*, Pekka Abrahamsson, Andreas Jedlitschka, Anh Nguyen Duc, Michael Felderer, Sousuke Amasaki, and Tommi Mikkonen (Eds.). Springer International Publishing, Cham, 590–597.

[27] Per Runeson, Emelie Engström, and Margaret-Anne D. Storey. 2020. The Design Science Paradigm as a Frame for Empirical Software Engineering. In *Contemporary Empirical Methods in Software Engineering*.

[28] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Softw. Engg.* 14, 2 (apr 2009), 131–164.

[29] SonarSource S.A. [n. d.]. SonarQube. https://www.sonarsource.com/products/sonarqube/

[30] SAFECode. 2018. Fundamental practices for secure software development. https://safecode.org/wp-content/uploads/2018/03/SAFECode_Fundamental_Practices_for_Secure_Software_Development_March_2018.pdf

[31] Phillip Schneider, Markus Voggenreiter, Abdullah Gulraiz, and Florian Matthes. 2022. Semantic Similarity-Based Clustering of Findings From Security Testing Tools. In *International Conference on Natural Language and Speech Processing*.

[32] Carolyn B. Seaman. 2008. *Qualitative Methods*. Springer London, London, 35–62.

[33] Klaas-Jan Stol and Brian Fitzgerald. 2020. Guidelines for Conducting Software Engineering Research. In *Contemporary Empirical Methods in Software Engineering*, Michael Felderer and Guilherme Horta Travassos (Eds.). Springer International Publishing, Cham, 27–62.

[34] Richard J. Thomas, Joseph Gardiner, Tom Chothia, Emmanouil Samanis, Joshua Perrett, and Awais Rashid. 2020. Catch Me If You Can: An In-Depth Study of CVE Discovery Time and Inconsistencies for Managing Risks in Critical Infrastructures. In *Proceedings of the 2020 Joint Workshop on CPS & IoT Security and Privacy* (Virtual Event, USA) *(CPSIOTSEC'20)*. Association for Computing Machinery, New York, NY, USA, 49–60. https://doi.org/10.1145/3411498.3419970

[35] Kennedy A. Torkura, Muhammad I.H. Sukmana, and Christoph Meinel. 2017. Integrating Continuous Security Assessments in Microservices and Cloud Native Applications. In *Proceedings of The10th International Conference on Utility and Cloud Computing* (Austin, Texas, USA) *(UCC '17)*. Association for Computing Machinery, New York, NY, USA, 171–180. https://doi.org/10.1145/3147213.3147229

[36] Markus Voggenreiter. 2023. Supplementary Material. (10 2023). https://doi.org/10.6084/m9.figshare.24235147.v1

[37] Markus Voggenreiter and Ulrich Schöpp. 2022. Using a Semantic Knowledge Base to Improve the Management of Security Reports in Industrial DevOps Projects. *2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (2022), 309–310.

[38] Markus Voggenreiter and Ulrich Schöpp. 2023. Prioritizing Industrial Security Findings in Agile Software Development Projects. 375–379. https://doi.org/10.1109/ICSE-Companion58688.2023.00106

[39] Stefan Wagner, Daniel Méndez Fernández, Michael Felderer, Daniel Graziotin, and Marcos Kalinowski. 2020. Challenges in Survey Research. In *Contemporary Empirical Methods in Software Engineering*.

[40] Ju An Wang and Minzhe Guo. 2009. OVM: An Ontology for Vulnerability Management. In *Proceedings of the 5th Annual Workshop on Cyber Security and Information Intelligence Research: Cyber Security and Information Intelligence Challenges and Strategies* (Oak Ridge, Tennessee, USA) *(CSIIRW '09)*. Association for Computing Machinery, New York, NY, USA, Article 34, 4 pages. https://doi.org/10.1145/1558607.1558646

[41] Anna Wiedemann, Manuel Wiesche, and Helmut Krcmar. 2019. Integrating Development and Operations in Cross-Functional Teams - Toward a DevOps Competency Model *(SIGMIS-CPR '19)*. Association for Computing Machinery, New York, NY, USA, 14–19. https://doi.org/10.1145/3322385.3322400

[42] Shahed Zaman, Bram Adams, and Ahmed E. Hassan. 2011. Security versus Performance Bugs: A Case Study on Firefox. In *Proceedings of the 8th Working Conference on Mining Software Repositories* (Waikiki, Honolulu, HI, USA) *(MSR '11)*. Association for Computing Machinery, New York, NY, USA, 93–102.